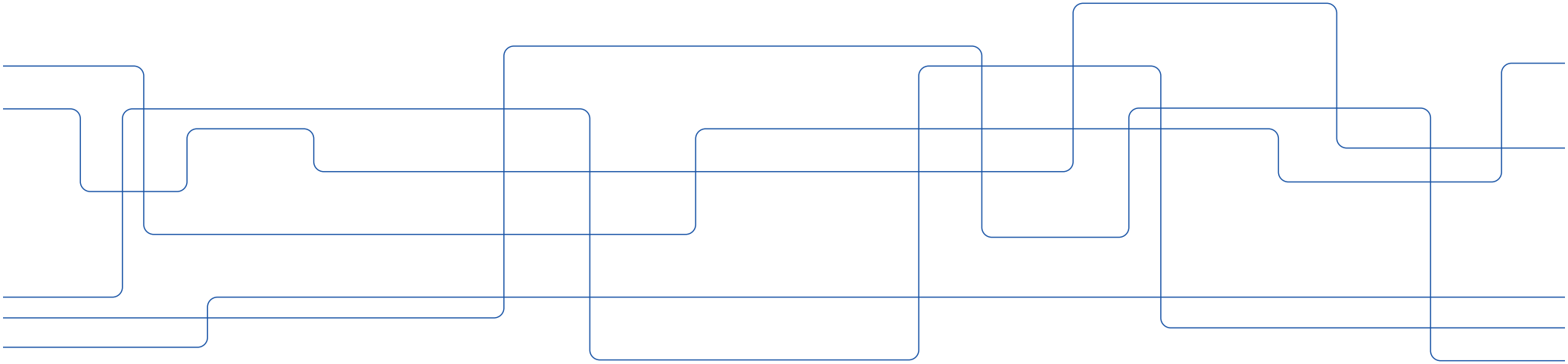


HI1024

Grundkurs i programmering

F13: Filer (FILE)!



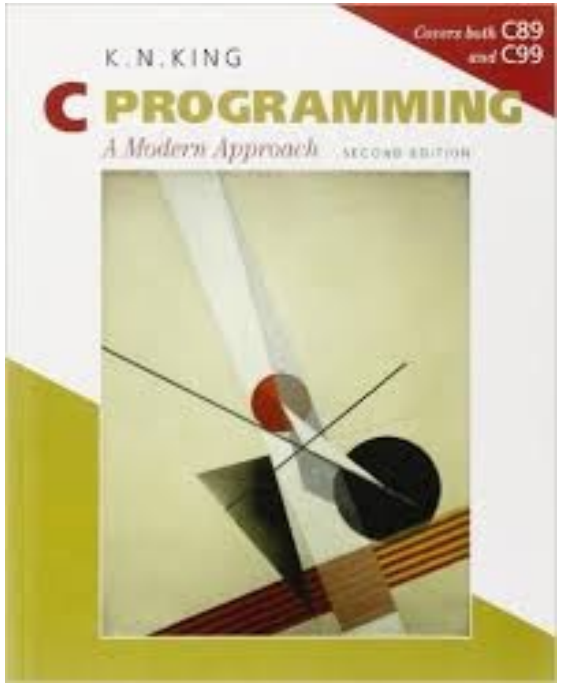
Innehåll!

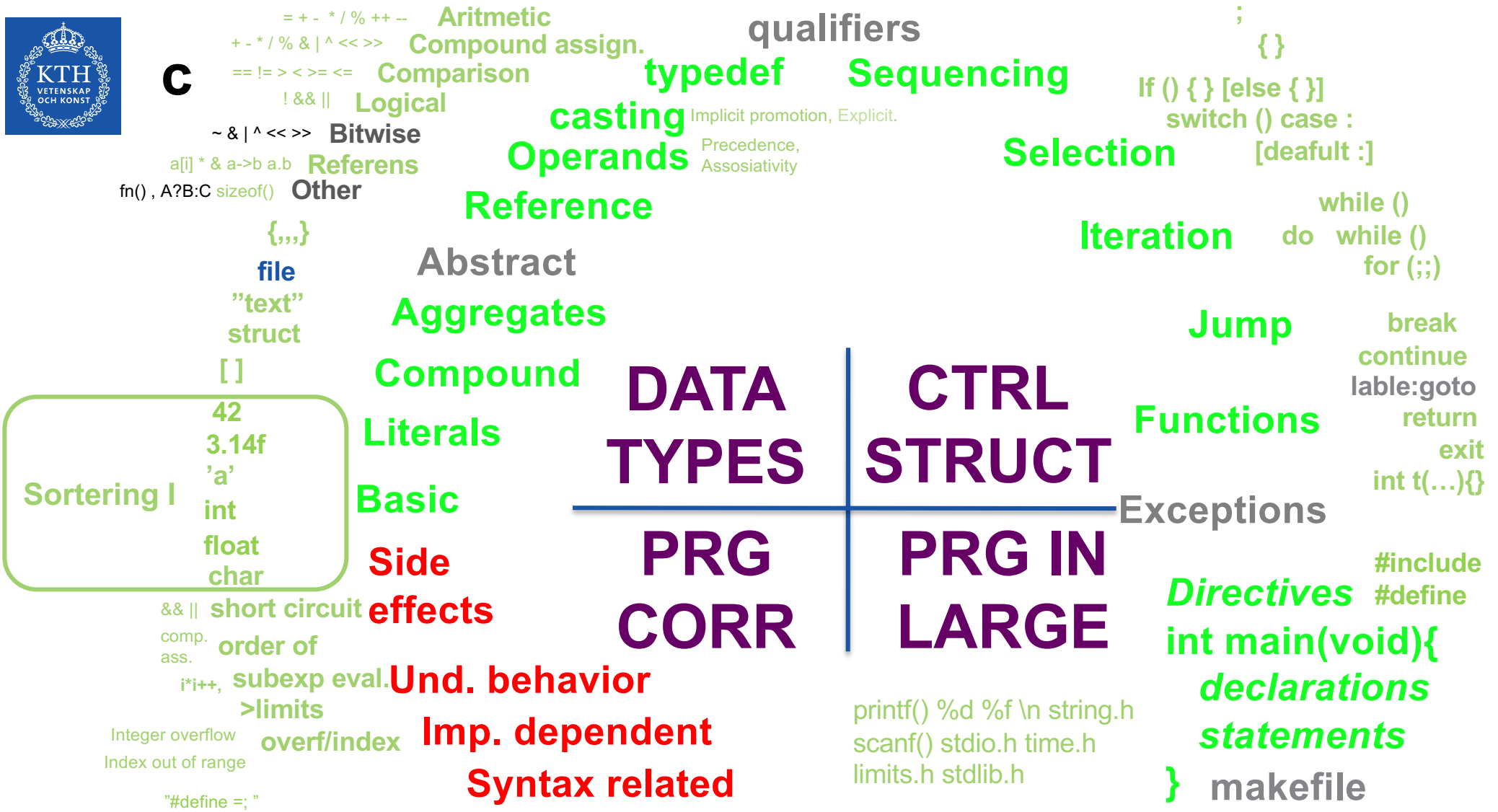
King 22.1..5

Repetition föreläsning #12

Filer

Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
file
”text”
struct
[ ]
Sortering I
42
3.14f
’a’
int
float
char
Literals
Basic
Compound
Abstract
Aggregates
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
Iteration
do while ()
for (;;)
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
Index out of range
overf/index
Und. behavior
Imp. dependent
Syntax related
”#define =; ”
printf() %d %f \n string.h
scanf() stdio.h time.h
limits.h stdlib.h
int/int
/*comment*/ or //

Poster (struct)

WMM699 **Scoda**

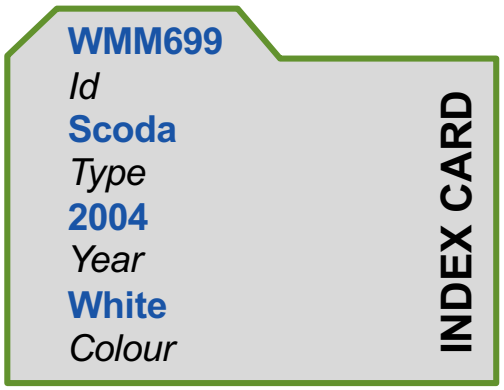
Id **Type**



Year **Color**

2004 **White**

…vi behöver en struct! Med hjälp av det reserverade ordet struct går det att skapa en ny datatyp som kan innehålla de variabler som önskas. Variablerna kallas members och utgör en egen namnrymd…



Stor bokstav (typ)

```
struct Car {
    char id[7];
    char type[30];
    int  year;
    char color[10];
};

struct Car myCar;

myCar.year = 2004;
```

…skapa en variable av den sorten via struct Car, och nå enskilda medlemmar via punktnotation!

Poster (struct)

Det finns givetvis ett sätt att initiera en hel post!

```
struct Car myFastCar = {"WOU039", "MAZDA", 2015, "RED"};
struct Car mySlowCar = {"XBG678", "SCODA", 2004, "BLUE"};
```

== & != fungerar **inte** på poster (eller matriser) **men**
= fungerar på poster* (till skillnad från matriser)!!!!!!!

```
struct Car carToSell;

carToSell = mySlowCar;   // Wow, all elements copied!
```

* Båda posterna måste vara av samma typ! D.v.s om någon skapat struct Bil b1 som är exakt lika struct Car b2 (samma medlemmar) går det inte att göra b1=b2.

Poster (struct)

- Det är lätt att glömma ”struct”-delen när en postvariabel ska skapas, med typedef går det att skapa en helt egen datatyp på följande sätt:

```
typedef struct {
    char id[6];
    char type[30];
    int  year;
    char color[10];
} Car;

Car myFastCar = {"WOU039", "MAZDA", 2015, "RED"};
Car mySlowCar = {"XBG678", "SCODA", 2004, "BLUE"};

Car carToSell;

carToSell = mySlowCar;   // Wow, all elements copied!
```

- Observera att det nya typnamnet ska stå efter struct-def!

Nya typer definieras med fördel direct efter #-direktiv, i början av filen, så att de “syns” i hela filen!

Testprogram…

Riktigt bra blir det om du använder den nya operatorn ’->’ som är en kombination av * och . med prioriteten rätt så att:

(*pCar).id kan skrivas smidigt som **pCar->id**

```
Garage 0.4
REG NB : WOU039
TYPE   : MAZDA
YEAR   : 2015
COLOR  : RED
Garage exit!
```

BÄST!

```c
#include <stdio.h>
#include <string.h>

// A Car package ---------------------------------
#define IDSIZE 6
#define TYPESIZE 30
#define COLORSIZE 10

typedef struct {
    char id[IDSIZE+1];
    char type[TYPESIZE+1];
    int  year;
    char color[COLORSIZE+1];

} Car;

void printCar(Car *pCar){
    printf("REG NB : %s\n", pCar->id);
    printf("TYPE   : %s\n", pCar->type);
    printf("YEAR   : %d\n", pCar->year);
    printf("COLOR  : %s\n", pCar->color);
}
// End of Car package ----------------------------

int main(int argc, char **argv)
{
    Car myFastCar = {"WOU039", "MAZDA", 2015, "RED"};

    printf("Garage 0.4\n");

    printCar(&myFastCar);

    printf("Garage exit!\n");
    return 0;
}
```

Ny operator ->

Poster

Funktioner kan returnera en post…

…det kan tyckas otroligt märkligt då variabler skapade i en funktion bara finns så länge funktionen finns…

…men eftersom alla parametrar hanteras via copy-by-value så

fungerar det!

```
Garage 0.5
REG NB : WOU039
TYPE   : MAZDA
YEAR   : 2015
COLOR  : RED
Garage exit!
```

```c
#define TYPESIZE 30
#define COLORSIZE 10

typedef struct {
    char id[TYPESIZE];
    char type[TYPESIZE];
    int  year;
    char color[COLORSIZE];
} Car;

void printCar(Car *pCar){
    printf("REG NB : %s\n", pCar->id);
    printf("TYPE   : %s\n", pCar->type);
    printf("YEAR   : %d\n", pCar->year);
    printf("COLOR  : %s\n", pCar->color);
}

Car initCar(char itsId[], char itsType[],
            int itsYear, char itsColor[]){
    Car newCar;
    strcpy(newCar.id,    itsId);
    strcpy(newCar.type,  itsType);
           newCar.year = itsYear;
    strcpy(newCar.color, itsColor);
    return newCar;
}
// End of Car package ------------------------

int main(int argc, char **argv)
{
    Car myFastCar = initCar("WOU039", "MAZDA",
                            2015, "RED");

    printf("Garage 0.5\n");

    printCar(&myFastCar);

    printf("Garage exit!\n");
    return 0;
}
```


(Skapa alltid en init- & print-funktion för dina poster!)

Poster (Överkurs)

Egentligen vill vi inte att någon ska veta hur id, type, årsmodell & färg är lagrade i en variabel av Car typ…

...vi har valt en struct men det hade också fungerat med en 2-dim array av char, etc etc etc...

...det är svårt att dölja bort det, men manipulation av informationen skulle kunna ske via speciella anrop!

Det är vanligt att skapa **accessor** funktioner för att hämta info, **mutator** funktioner för att ändra info eller **manager** funktioner för t.ex. initiering!





Poster & Matriser

På samma sätt som…

```
int myInt;                  //en variabel för en int.
int myIntTable[3];          //en matris för 3 int...
```

…går det att…

```
Car theCar;                 // en variabel för en bilpost.
Car theCarDB[6];            // en matris för 6 bilposter!
                            // access via t.ex: theCarDB[2].year
```

Nytt: Poster & Matriser

Skapa en variabel som har plats för 3 bilar…

…initiera 2 platser, det behövs ofta också en variabel som talar om hur många element i matrisen som är använda…

…nu hade vi nog behövt en printGarage funktion!

(Början på ett Garagepaket!)

```
// A Garage package ------------------------------
#define GARAGESIZE 3

// End of Garage package -------------------------

int main(int argc, char **argv)
{
    Car myGarage[GARAGESIZE]; // My garage...
    int used = 0;             // ...currently empty!

    myGarage[0] = initCar("WOU039", "MAZDA", 2015, "RED");
    myGarage[1] = initCar("XDW693", "SCODA", 2004, "BLUE");
    used = 2;

    printf("Garage 0.6\n");

    for (int i=0; i<used; i++)
        printCar(&myGarage[i]);

    printf("Garage exit!\n");
    return 0;
}
```

```
Garage 0.6
REG NB : WOU039
TYPE   : MAZDA
YEAR   : 2015
COLOR  : RED
REG NB : XDW693
TYPE   : SCODA
YEAR   : 2004
COLOR  : BLUE
Garage exit!
```

Övning

- **Skapa ett bokregister…**

- …Som kan hantera 100 böcker och
- …Har en ”huvudkommandoloop” där
- ……’a’ registrerar en (1) ny bok och
- ……’v’ skriver ut alla registrerade böcker och
- ……’q’ avslutar programmet!

- (Utan globala variabler, inga feltester!)

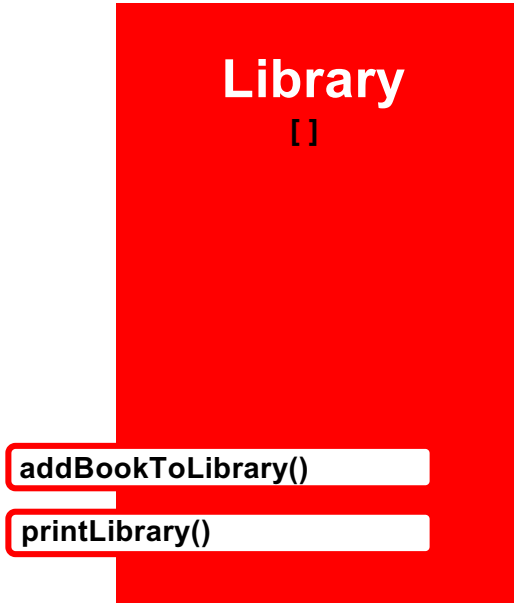
```
Library 1.0
warning: this program use
vaq:a
Title  ? Winnie the Pooh
Author ? A. A. Milnes
Price  ? 0.75
Where  ? Library
vaq:v
TITLE  : Winnie the Pooh
AUTHOR : A. A. Milnes
PRICE  : 0.75
WHERE  : Library
vaq:q
Library exit!
```

Övning: Systemarkitektur – Top Down!

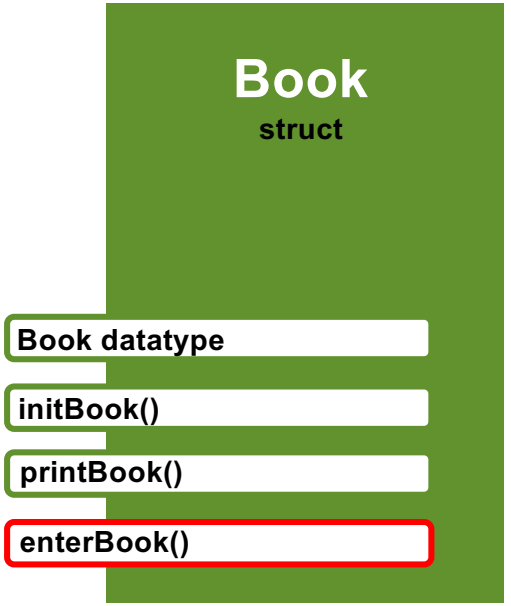
Om vi har ett anv.gränssnitt...

```
Library 1.0
warning: this program use
vaq:a
Title  ? Winnie the Pooh
Author ? A. A. Milnes
Price  ? 0.75
Where  ? Library
vaq:v
TITLE  : Winnie the Pooh
AUTHOR : A. A. Milnes
PRICE  : 0.75
WHERE  : Library
vaq:q
Library exit!
```

…och vi hade ett bibliotekspaket…



…byggt på vårt bokpaket…



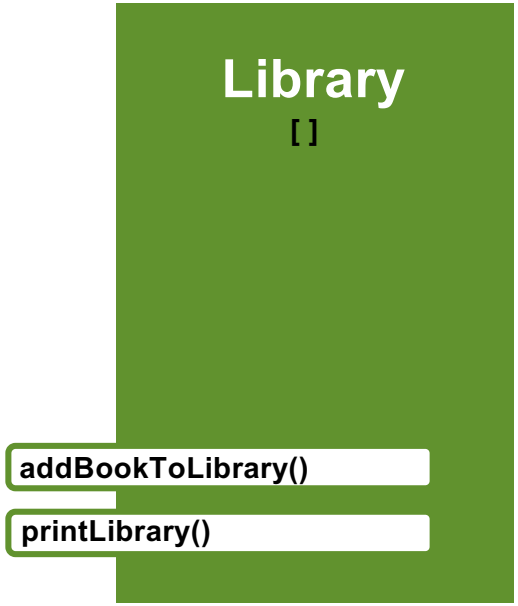
…så skulle vi vara nära mål!

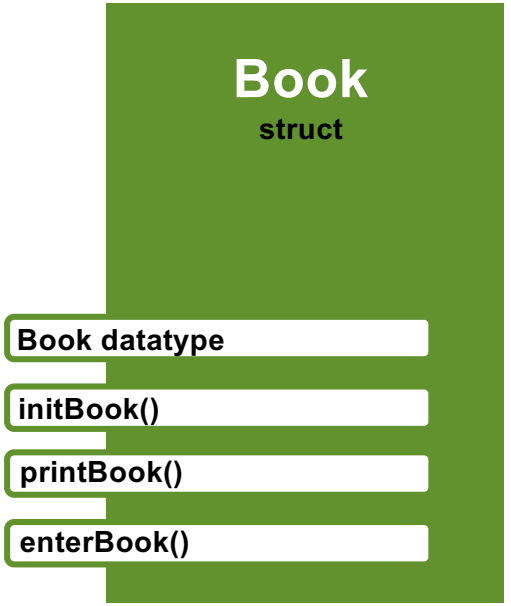
I detta fall kan vi vänta med användardialogen och först arbeta lite med bibliotekspaketet…

Övning

Wow, vi har våra enhetstestade paket, dags att fixa till användargränssnittet!

```
Library 1.0
warning: this program use
vaq:a
Title  ? Winnie the Pooh
Author ? A. A. Milnes
Price  ? 0.75
Where  ? Library
vaq:v
TITLE  : Winnie the Pooh
AUTHOR : A. A. Milnes
PRICE  : 0.75
WHERE  : Library
vaq:q
Library exit!
```





I vanliga fall hade det varit smart att göra det först, men paketen var lätt-testade, så det gick att utveckla dem utan användargränssnittet!

Övning

```
Library 1.0
warning: this program use
vaq:a
Title  ? Winnie the Pooh
Author ? A. A. Milnes
Price  ? 0.75
Where  ? Library
vaq:v
TITLE  : Winnie the Pooh
AUTHOR : A. A. Milnes
PRICE  : 0.75
WHERE  : Library
vaq:q
Library exit!
```

```c
int main(int argc, char **argv) {
    Book myLibrary[LIBSIZE];
    int  books=0;
    char cmd;
    printf("Library 1.0\n");

    do {
        printf("vaq:");
        scanf("%c%*c", &cmd);      // Must remove '\n'
        switch (cmd) {
           case 'a' : addBookToLibrary(myLibrary,
                                       &books,
                                       enterBook());
                      break;
           case 'v' : printLibrary(myLibrary, books);
                      break;
        }
    } while (cmd!='q');

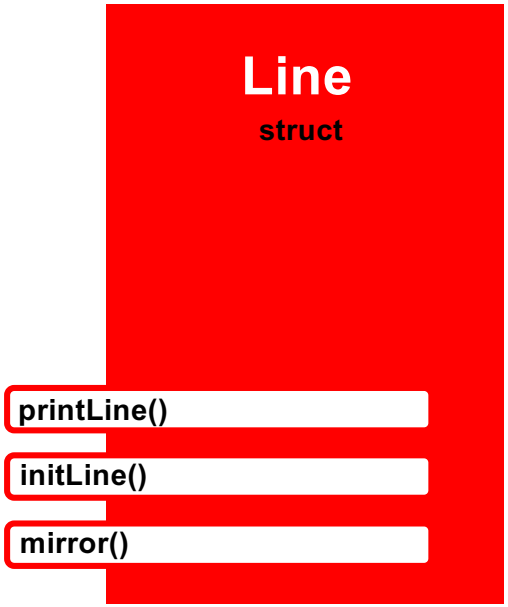
    printf("Library exit!\n");
    return 0;
}
```

Övning: Linjebibliotek

```
Line Toolbox!
y=-2.5x+1.5
y=2.5x+1.5
LT Done, bye!
```

Du vet säkert att en rät linje kan beskrivas som y=kx+m.

1. Skapa en typ Line med elementen k & m (float).
2. Skapa en funktion printLine( ) som via en pekare skriver ut en Line.
3. Skapa en funktion ”initLine(k,m)” som returnerar Line.
4. Skapa en funktion mirror( ) som utför k*-1.



Övning: Linjebibliotek

```
Line Toolbox!
y=-2.5x+1.5
y=2.5x+1.5
LT Done, bye!
```

Du vet säkert att en rät linje kan beskrivas som y=kx+m.

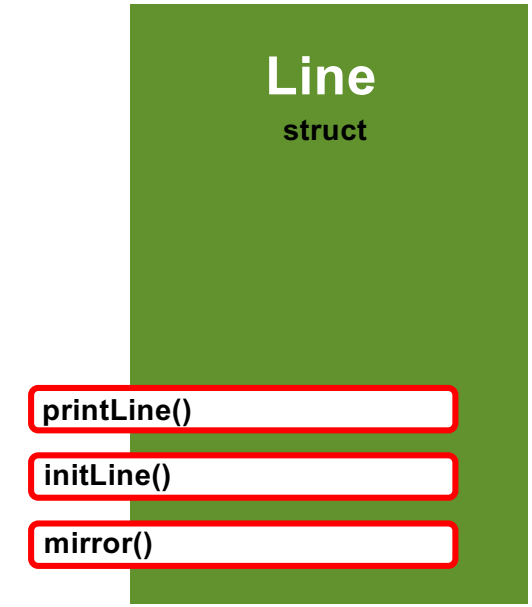
1. Skapa en typ Line med elementen k & m (float).

```
typedef struct {
    float k,m;
} Line;
```

2. Skapa en funktion printLine( ) som via en pekare skriver ut en Line.

3. Skapa en funktion ”initLine(k,m)” som returnerar Line.

4. Skapa en funktion mirror( ) som utför k*-1.



Övning: Linjebibliotek

```
Line Toolbox!
y=-2.5x+1.5
y=2.5x+1.5
LT Done, bye!
```

Du vet säkert att en rät linje kan beskrivas som y=kx+m.

1. Skapa en typ Line med elementen k & m (float).

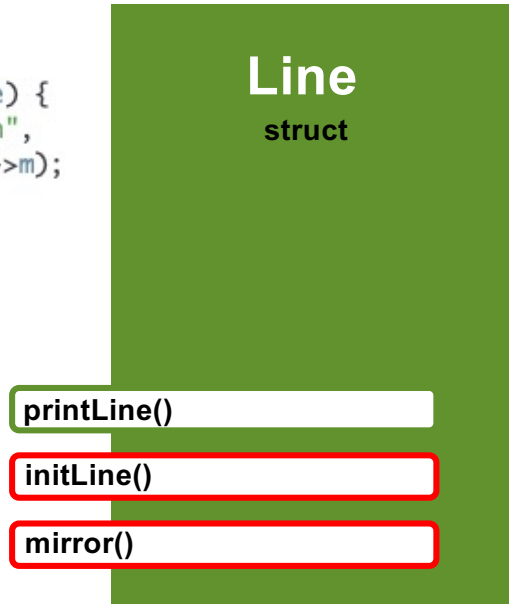
2. Skapa en funktion printLine( ) som via en pekare skriver ut en Line.

3. Skapa en funktion ”initLine(k,m)” som returnerar Line.

4. Skapa en funktion mirror( ) som utför k*-1.

```c
typedef struct {
    float k,m;
} Line;
```

```c
void printLine(Line *pLine) {
    printf("y=%.1fx+%.1f\n",
           pLine->k,pLine->m);
}
```



Övning: Linjebibliotek

```
Line Toolbox!
y=-2.5x+1.5
y=2.5x+1.5
LT Done, bye!
```

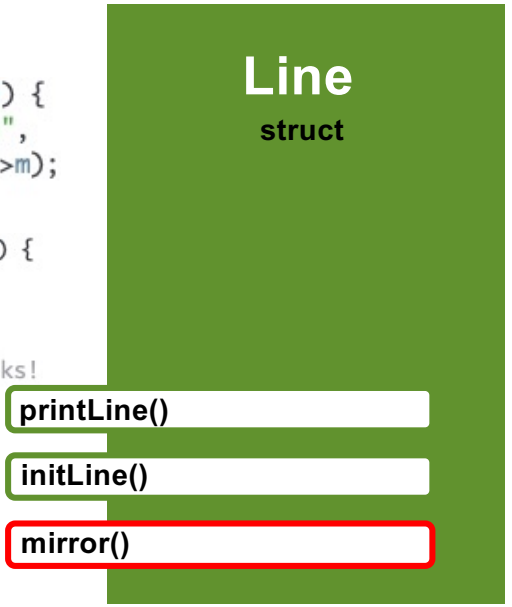
Du vet säkert att en rät linje kan beskrivas som y=kx+m.

1. Skapa en typ Line med elementen k & m (float).
2. Skapa en funktion printLine( ) som via en pekare skriver ut en Line.
3. Skapa en funktion ”initLine(k,m)” som returnerar Line.
4. Skapa en funktion mirror( ) som utför k*-1.

```c
typedef struct {
    float k,m;
} Line;

void printLine(Line *pLine) {
    printf("y=%.1fx+%.1f\n",
           pLine->k,pLine->m);
}

Line initLine(float k, float m) {
    Line newLine;
    newLine.k = k;
    newLine.m = m;
    //return (Line){k, m}; Works!
    return newLine;
}
```



Övning: Linjebibliotek

```
Line Toolbox!
y=-2.5x+1.5
y=2.5x+1.5
LT Done, bye!
```

Du vet säkert att en rät linje kan beskrivas som y=kx+m.

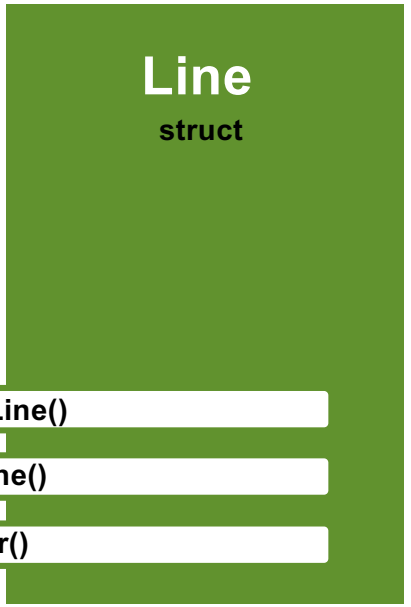
1. Skapa en typ Line med elementen k & m (float).
2. Skapa en funktion printLine( ) som via en pekare skriver ut en Line.
3. Skapa en funktion ”initLine(k,m)” som returnerar Line.
4. Skapa en funktion mirror( ) som utför k*-1.

```
typedef struct {
    float k,m;
} Line;

void printLine(Line *pLine) {
    printf("y=%.1fx+%.1f\n",
           pLine->k,pLine->m);
}

Line initLine(float k, float m) {
    Line newLine;
    newLine.k = k;
    newLine.m = m;
    //return (Line){k, m}; Works!
    return newLine;
}

void mirror(Line *pLine){
    pLine->k=pLine->k*(-1);
}
```



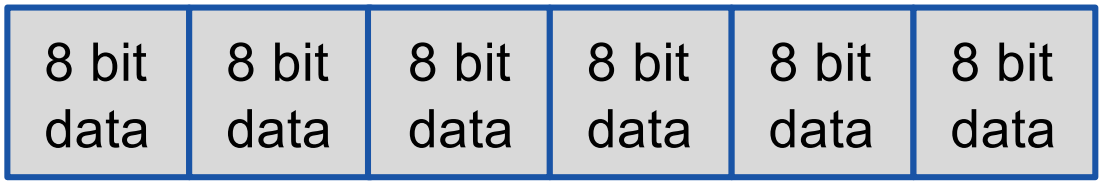


Nytt: Filer!

Det är lite surt att förra föreläsningens bil- och bokregister inte kunde ”spa-ras”, när progr. avslutades förlorades alla uppgifter som registrerats…

…givetvis går det att ”spara” registren som en fil på hårddisken och i dag ska vi lära oss hur…

…via har i smyg redan arbetat med filer, C-program ser på bildskärmen och tangentbordet som filer! En fil är enbart en sekvens, ström, följd (stream) av data, ofta med en början och ett slut:



Nytt: Textfiler vs Binärfiler

Det första beslutet vi måste ta är hur informationen i filen ska sparas. Tänk er att vi vill lagra talen 128 och 255. Då skulle vi antingen kunna skriva in siffrorna…

1	2	8		2	5	5

…eller så inser vi att talen ”ryms” i 8-bitar och skriver binärt…

1000 0000	1111 1111

…det första kallas för en **textfil** och det senare för en **binärfil**.

Binärfilen tar mindre plats men är svårare att läsa, används för specialfall, **kursen använder textfiler!**

Nytt: Filer

Det är inte svårare att skriva och läsa till en fil än att skriva och läsa från bildskärm och tangentbord, men programmet måste skaffa sig ett ”handtag” till filen!

”Handtaget” har datatypen FILE och ska användas för att skapa en pekare till filen.

”Handtaget” etableras via anropet:

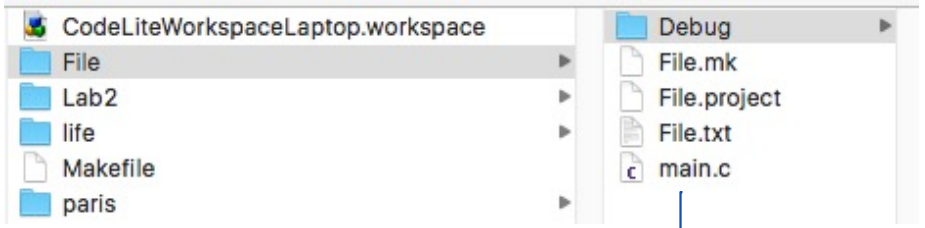
```
FILE *fopen(const char *filename, const char *mode)
```

Vad filen heter: ”register.txt”

mode	Description
"r"	Opens a file for reading. The file must exist.
"w"	Creates an empty file for writing. If a file with the same name already exists, its content is erased and the file is considered as a new empty file.
"a"	Appends to a file. Writing operations, append data at the end of the file. The file is created if it does not exist.

(Textfil är default!)
((mode b = binär))

Nytt: Filer - Skapa



4 rader!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    FILE *pFile;
    printf("File test 0.1\n");

    if ((pFile=fopen("data.txt","w")))
        fprintf(pFile, "Hello World 2015\n");
    else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

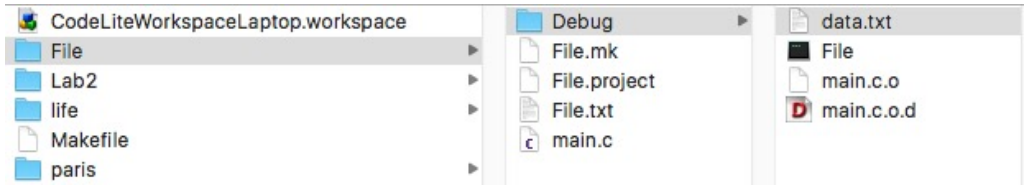
```
* File test 0.1
* File test done!
```

Ny fil!

(Fanns en data.txt så raderades den och sedan skapades den nya filen!)

data.txt





(Det går att ange ett så kallat path före filnamnet, för att skapa filen på en annan plats, men det ingår inte i kursen.)

Nytt: Filer - Skapa

Tänk på att det är otroligt viktigt att skriva ut ”ny rad”, om du vill ha en ny rad, på samma sätt som om du skulle läst innehållet på bildsk, i annat fall skrivs allt samman till en rad i filen!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    FILE *pFile;
    printf("File Test 0.1\n");

    if ((pFile=fopen("data.txt","w")))
        fprintf(pFile, "Hello World 2015\n");
    else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

```c
#include <stdio.h>

int main(int argc, char **argv) {
    FILE *pFile;
    printf("File Test 0.1\n");

    if ((pFile=fopen("data.txt","w"))) {
        fputs("Hello World 2015\n",pFile);
    } else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

Det finns (givetvis) en fputs också men: 1) se upp med ordningen på argumenten och 2) den skriver inte ut ett avslutande ’\n’ som puts gör!

(Om det som ska skrivas ut är enbart en sträng så är fputs() ”effektivare” än fprintf())

Nytt: Filer – Trick #1

Bildskärmen och tangentbordet finns alltid tillgängliga via variablerna **std-out** och **stdin** (om du har stdio.h inkluderat)! Under utveckling, testa med **stdout,** så kommer allt ut på skärmen som det skulle ha hamnat i filen!!!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    FILE *pFile;
    printf("File Test 0.1\n");

    if ((pFile=fopen("data.txt","w"))) {
        fputs("Hello World 2015\n",pFile);
    } else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

```
* File test 0.1
* File test done!
```

```c
#include <stdio.h>

int main(int argc, char **argv) {
    FILE *pFile;
    printf("File Test 0.1\n");

    if ((pFile=fopen("data.txt","w"))) {
        fputs("Hello World 2015\n",stdout);
    } else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

```
File Test 0.1
Hello World 2015
File Test done!
```

Nytt: Filer - Läsa

fscanf() fungerar på samma sätt (och med samma problem) som scanf()!

fscanf() returnerar **antalet inlästa argument** - är inte det det förväntade antalet är filen troligen slut…

Vi försöker läsa tillbaks texten i filen data.txt…

```
* File test 0.2
[1][Hello]
[1][World]
[1][2015]
[-1][2015]
* File test done!
```

(Hello World 2015\n)

(%s läser bort inledande ws därför ”hittar” den inget sista gången)

```c
#include <stdio.h>

int main(int argc, char **argv) {
    FILE *pFile;
    int  count;
    char msg[30];
    printf("File Test 0.2\n");

    if ((pFile=fopen("data.txt","r"))) {
        count=fscanf(pFile, "%s",msg);
        printf("[%d][%s]\n",count, msg);
        count=fscanf(pFile, "%s",msg);
        printf("[%d][%s]\n",count, msg);
        count=fscanf(pFile, "%s",msg);
        printf("[%d][%s]\n",count, msg);
        count=fscanf(pFile, "%s",msg);
        printf("[%d][%s]\n",count, msg);
    } else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

4 rader!

Det behövs (nästan) samma 4 magiska rader för att det ska fungera!

Nytt: Filer - Läsa

Precis som tidigare är det matchningssträngen som avgör hur scanf försöker tolka innehållet!

```c
/*File 0.3 */
#include <stdio.h>
#include <string.h>
#define STRSIZE 10

int main(void) {
   FILE *dataPtr;
   int  count, year;
   char msg1[STRSIZE], msg2[STRSIZE];

   printf("* File test 0.3\n");

   if (dataPtr=fopen("/F12/data.txt","r")) {
      count=fscanf(dataPtr,"%s %s %d",
                   msg1, msg2, &year);
      printf("[%d][%s][%s][%d]\n",
             count, msg1, msg2, year);
   } else {
      printf("Error: Open file!");
   }

   fclose(dataPtr);
   printf("* File test done!\n");
   return 0;
}
```

```
* File test 0.3
[3][Hello][World][2015]
* File test done!
```

```c
#include <stdio.h>

int main(int argc, char **argv) {
    FILE *pFile;
    int  count;
    char msg[30];
    printf("File Test 0.3\n");

    if ((pFile=fopen("data.txt","r"))) {
        count=fscanf(pFile, "%[^\n]%*c", msg);
        printf("[%d] [%s]\n", count, msg);
    } else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

```
File Test 0.3
[1] [Hello World 2015]
File Test done!
```

OBS: %* matcningar räknas inte!

Nytt: Filer - Läsa

```c
#include <stdio.h>
#define MAXSTR 30

int main(int argc, char **argv) {
    FILE *pFile;
    char msg[MAXSTR];
    printf("File Test 0.4\n");

    if ((pFile=fopen("data.txt","r"))) {
        if (fgets(msg, MAXSTR, pFile))
            printf("[%s]\n", msg);
        else
            printf("NULL ptr - EOF\n");
    } else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

```
File Test 0.4
[Hello World 2015
]
File Test done!
```

stdio.h har haft en gets funktion som läser in en hel rad, men den kunde lätt skriva sönder minnet, och har tagits bort…

…däremot finns fgets som är verkligt bra, läser in tecken enligt 1..3 & lägger till '\0':

1) Läser in maximalt MAXSTR-1 tecken
2) Läser in, men stannar, efter ett '\n'
3) Stannar om den kommer till filens slut

(returnerar pekare till strängen, eller NULL om EOF och inget tecken läst!)

Trick: Använd filpekaren **stdin** för att läsa in från tangetbordet med kontroll!

Nytt: Filer - Läsa

```c
#include <stdio.h>
#define MAXSTR 30

int main(int argc, char **argv) {
    FILE *pFile;
    char msg[MAXSTR];
    int  ch, cnt=0;
    printf("File Test 0.5\n");

    if ((pFile=fopen("data.txt","r"))) {
        while ((ch=getc(pFile)) != EOF)
            msg[cnt++]=(char)ch;
        msg[cnt]='\0';
        printf("[%d][%s]\n", cnt, msg);
    } else
        printf("* ERROR: Open file!\n");
    fclose(pFile);

    printf("File Test done!\n");
    return 0;
}
```

stdio.h har också en funktion för att läsa enstaka tecken från en fil, getc()…

…tänk på att den returnera en int som är positionen för tecknet i ASCII-alfabetet eller EOF (som ofta är -1) om filen är slut!

Kom ihåg att lägga till ett ’\n’!

Trick: Använd att tilldelning ’=’ returnerar ett värde som är lika med högerledet!

(Det går lätt att göra koden säker genom att testa cnt i while-satsen!)



Sammanfatning

”Mönstermatchning”	”Text, hela rader”	”Tecken för tecken”
fprintf()	fputs()	fputc()
fscanf()	fgets()	fgetc()
Returnerar antalet lyckade matchningar, om ej förväntat=EOF!	Returnerar NULL ptr vid EOF!	Returnerar EOF vid EOF!
Trick	**Trick**	**Trick**
stdout för debug!	fgets med **stdin**!	**=** är en operator!

Dags att spara bilarna i garaget…

```c
int main(int argc, char **argv)
{
    Car myGarage[GARAGESIZE]; // My garage...
    int used = 0;             // ...currently empty!

    myGarage[0] = initCar("WOU039", "MAZDA", 2015, "RED");
    myGarage[1] = initCar("XDW693", "SCODA", 2004, "BLUE");
    used = 2;

    printf("Garage 0.7\n");

    //myFastCar.year=2013;
    //setYear(&myFastCar, 2013);
    //printCar(&myFastCar);

    for (int i=0; i<used; i++)
       printCar(&myGarage[i]);

    saveGarage(myGarage, used);

    printf("Garage exit!\n");
    return 0;
}
```

```c
void saveGarage(Car aGarage[], int cars){
    FILE *pBF;
    printf("{Saving ");
    if ((pBF=fopen("data.txt","w")))
        for (int i=0; i<cars; i++) {
            fprintf(pBF, "%s %s %d %s\n",
                    aGarage[i].id,
                    aGarage[i].type,
                    aGarage[i].year,
                    aGarage[i].color);
            printf("*");
        }
    else
        printf("ERROR Could not save! ");
    fclose(pBF);
    printf(" }\n");
}
```

```
Garage 0.7
REG NB : WOU039
TYPE   : MAZDA
YEAR   : 2015
COLOR  : RED
REG NB : XDW693
TYPE   : SCODA
YEAR   : 2004
COLOR  : BLUE
{Saving ** }
Garage exit!
```

…och ladda tillbaks dem!

```c
int main(int argc, char **argv)
{
    Car myGarage[GARAGESIZE]; // My garage...
    int used = 0;             // ...currently empty!

    //myGarage[0] = initCar("WOU039", "MAZDA", 2015, "RED");
    //myGarage[1] = initCar("XDW693", "SCODA", 2004, "BLUE");
    //used = 2;

    printf("Garage 0.8\n");

    //myFastCar.year=2013;
    //setYear(&myFastCar, 2013);
    //printCar(&myFastCar);

    loadGarage(myGarage, &used);

    for (int i=0; i<used; i++)
       printCar(&myGarage[i]);

    saveGarage(myGarage, used);

    printf("Garage exit!\n");
    return 0;
}
```

```c
void loadGarage(Car aGarage[], int *pCars){
    FILE *pBF;
    printf("{Loading ");
    if ((pBF=fopen("data.txt","r")))
        while (fscanf(pBF, "%s %s %d %s",
                      aGarage[*pCars].id,
                      aGarage[*pCars].type,
                      &aGarage[*pCars].year,
                      aGarage[*pCars].color)==4) {
            (*pCars)++;
            printf("*");
        }
    else
        printf("ERROR Could not load! ");
    fclose(pBF);
    printf(" }\n");
}
```

```
Garage 0.8
{Loading ** }
REG NB : WOU039
TYPE   : MAZDA
YEAR   : 2015
COLOR  : RED
REG NB : XDW693
TYPE   : SCODA
YEAR   : 2004
COLOR  : BLUE
{Saving ** }
Garage exit!
```

Er tur, spara och läs tillbaks biblioteket!

```
Library 1.0
vaq:a
Title  ? Winnie the Pooh
Author ? A. A. Milnes
Price  ? 0.75
Where  ? Library
vaq:a
Title  ? Donald Duck
Author ? Carl Barks
Price  ? 1.25
Where  ? Safe
vaq:q
{Saving ** }
Library exit!
```

```
Winnie the Pooh
A.A. Milnes
0.750000
Library
Donald Duck
Carl Barks
1.250000
Safe
```

data.txt

```
Library 1.2
{Loading ** }
vaq:v
TITLE  : Winnie the Pooh
AUTHOR : A.A. Milnes
PRICE  : 0.75
WHERE  : Library
TITLE  : Donald Duck
AUTHOR : Carl Barks
PRICE  : 1.25
WHERE  : Safe
vaq:q
{Saving ** }
Library exit!
```


Tips: Innehållet i data.txt är sparat på ett litet annat sätt än för garaget, varför då?

Lab #3: Hantera Bank-konton!

```
(1) Registrera nya konton
(2) Skriva ut alla konton
(3) Sortera konton
(4) Soka efter konto
(5) Andra saldot på ett konto
(6) Avregistrera ett konto
(7) Statistik      (Missat Lab #1)
(8) Overforing     (Missat Lab #2)
(9) Avsluta
```

Konto-nr	Ägare	Saldo (kr)
000001	AFSHIN FARDI	1900
000002	AMRITPAL SINGH	276304
000003	PETER MAGNUSSON	5000

Lab #3: Hantera Bank-konton!

(0) Läs in konton från fil*

(1) Registrera nya konton
(2) Skriva ut alla konton
(3) Sortera konton
(4) Soka efter konto
(5) Andra saldot på ett konto
(6) Avregistrera ett konto
(7) Statistik (Missat Lab #1)
(8) Overforing (Missat Lab #2)
(9) Avsluta

(10) Spara konton till fil*

Konto-nr	Ägare	Saldo (kr)
000001	AFSHIN FARDI	1900
000002	AMRITPAL SINGH	276304
000003	PETER MAGNUSSON	5000

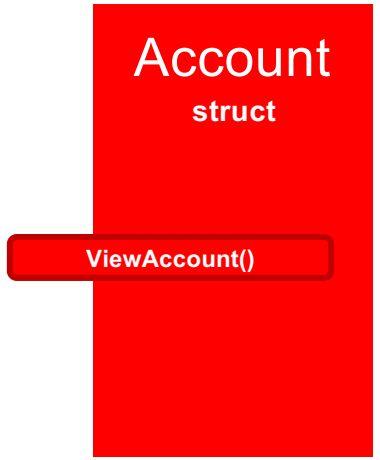
*Gör användargränssnittet först, underlättar testning! Gör 10) näst sist & 0) sist, du klarar dig långt med lite initieringar!

Lab #3: Hantera Bank-konton!

(0) Läs in konton från fil*

```
(1) Registrera nya konton
(2) Skriva ut alla konton
(3) Sortera konton
(4) Soka efter konto
(5) Andra saldot på ett konto
(6) Avregistrera ett konto
(7) Statistik    (Missat Lab #1)
(8) Overforing   (Missat Lab #2)
(9) Avsluta
```

(10) Spara konton till fil*



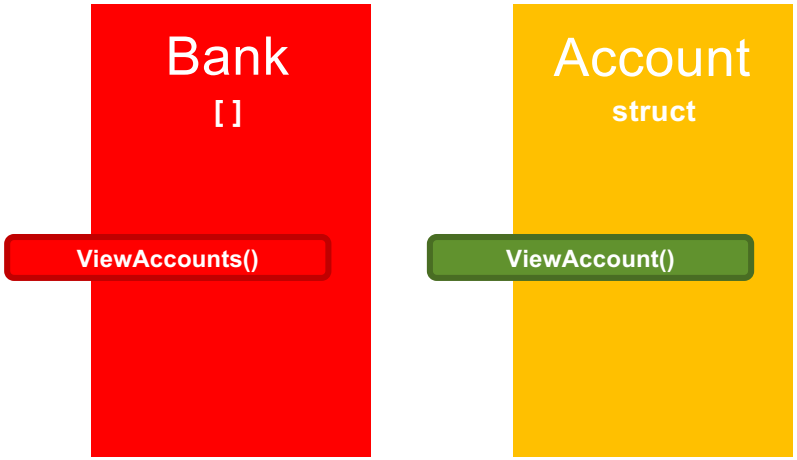
000001	AFSHIN FARDI	1900

Lab #3: Hantera Bank-konton!

(0) Läs in konton från fil*

(1) Registrera nya konton
(2) Skriva ut alla konton
(3) Sortera konton
(4) Soka efter konto
(5) Andra saldot på ett konto
(6) Avregistrera ett konto
(7) Statistik (Missat Lab #1)
(8) Overforing (Missat Lab #2)
(9) Avsluta

(10) Spara konton till fil*



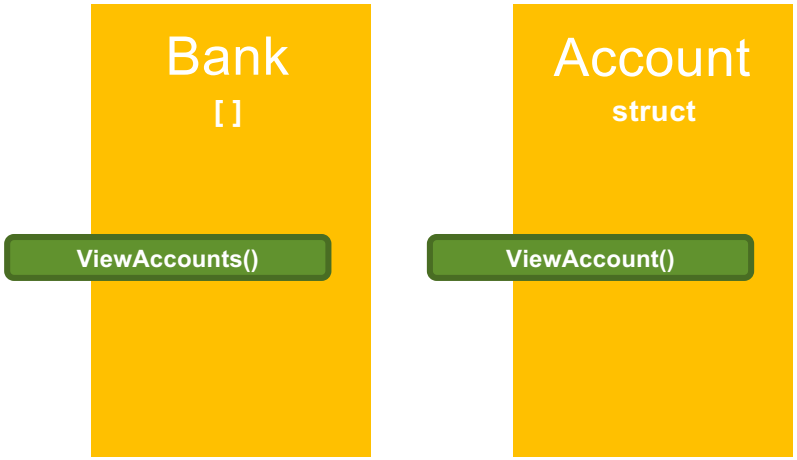
Konto-nr	Ägare	Saldo (kr)
000001	AFSHIN FARDI	1900
000002	AMRITPAL SINGH	276304
000003	PETER MAGNUSSON	5000

Lab #3: Hantera Bank-konton!

(0) Läs in konton från fil*

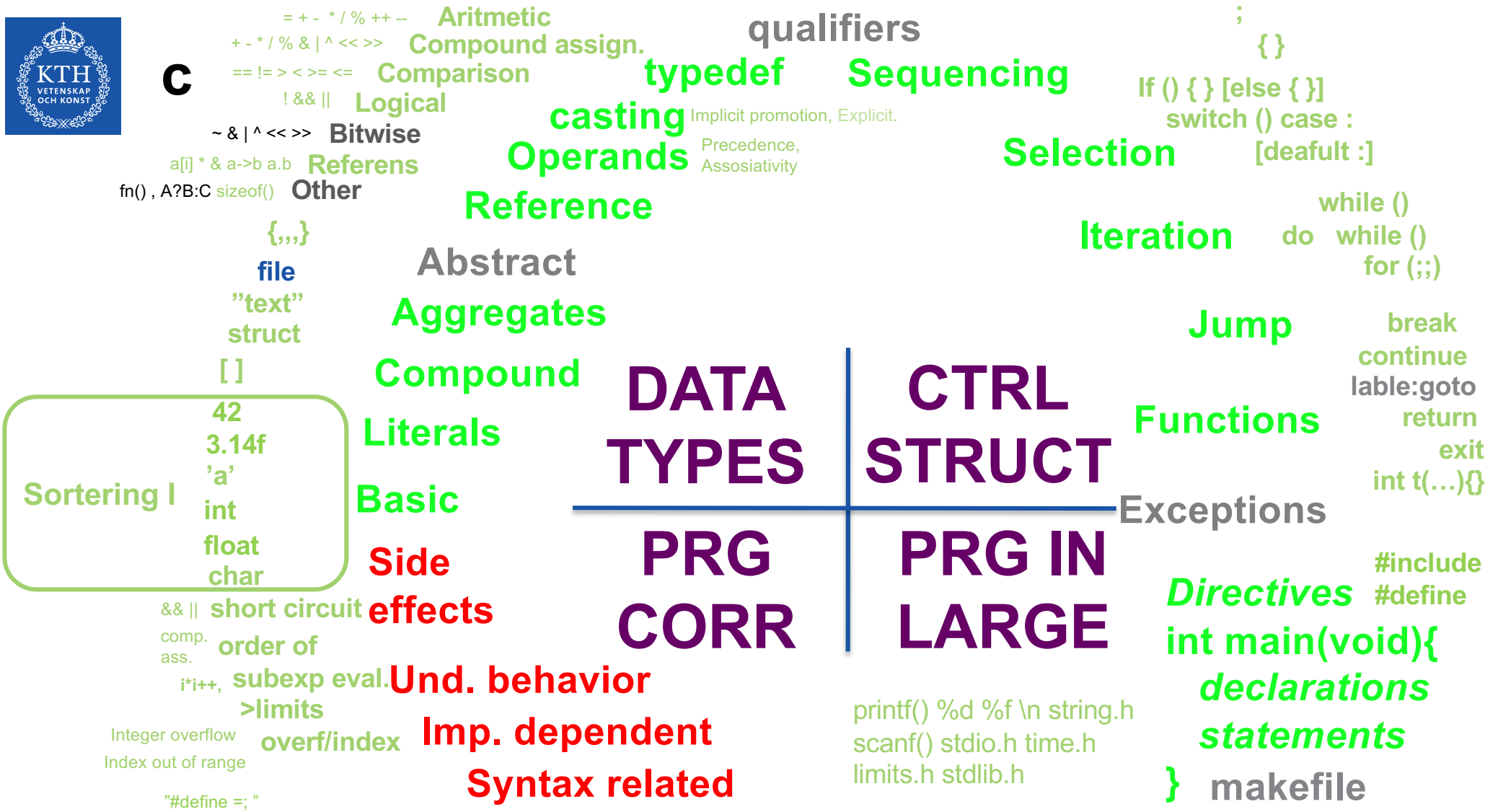
(1) Registrera nya konton
(2) Skriva ut alla konton
(3) Sortera konton
(4) Soka efter konto
(5) Andra saldot på ett konto
(6) Avregistrera ett konto
(7) Statistik (Missat Lab #1)
(8) Overforing (Missat Lab #2)
(9) Avsluta

(10) Spara konton till fil*



Konto-nr	Ägare	Saldo (kr)
000001	AFSHIN FARDI	1900
000002	AMRITPAL SINGH	276304
000003	PETER MAGNUSSON	5000

Arbeta vidare enligt tipsen i Lab PM! Kompilera ofta, och använd extra printf()!


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
file
”text”
struct
[ ]
Sortering I
42
3.14f
’a’
int
float
char
Literals
Basic
Compound
Abstract
Aggregates
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
Iteration
do while ()
for (;;)
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
Index out of range
overf/index
Side effects
Und. behavior
Imp. dependent
Syntax related
”#define =; ”
printf() %d %f \n string.h
scanf() stdio.h time.h
limits.h stdlib.h
int/int
/*comment*/ or //



Övn #13: Allt du inte gjort!

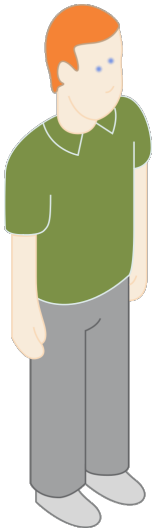
1 Skriv ner proble-
met så att ett barn
förstår det!

2 Gör ett minimalt
main() av dina
anteckningar!

3 Konstanter
Variabler med
Värde Uttryck
a..n & resultat!

4 Läs in riktiga
värden!

Historik



• 150817 0.1	AC	Skapad.
• 150821 1.0	AC	Övningsexempel .
• 160920 1.1	AC	Ny bättre övning.
• 170925 1.2	AC	Uppdaterad ny IDE.
• 190801 2.0	AC	Omarbetad från grunden, samt anpassad till KTH:s nya profil.